# Game Playing with Q-Learning on FrozenLake

AIAA3053 — Reinforcement Learning Principles and Methods
Final Project Report

Chi Kit WONG 50012338 | Haodong XU 50012601 | Sida ZHANG 50011956

**Abstract**

This report presents a complete Track A implementation project for the topic *Game Playing with Q-Learning*. We study tabular Q-Learning on the $4 \times 4$ FrozenLake task under both deterministic and stochastic transition dynamics, and we evaluate how the learning rate $\alpha$ influences convergence and final performance. The implementation supports a standard Gym-style interface and a native fallback environment so that the project remains reproducible even when external packages are unavailable. Across five random seeds, the deterministic environment converges to a 100.000 % final success rate, while the stochastic baseline reaches a mean final success rate of 0.658 with noticeably larger variance. An alpha ablation on the stochastic task shows that $\alpha = 0.50$ outperforms both $\alpha = 0.10$ and $\alpha = 0.01$, achieving the highest mean final success rate (0.696), the highest mean best success rate (0.782), and the fastest average convergence. The resulting figures, tables, and code provide an interpretable end-to-end reinforcement learning study suitable for reproducible coursework and later presentation.

## 1 Introduction

Reinforcement learning (RL) studies how an agent can learn decision-making behavior through interaction with an environment and scalar reward feedback. Unlike supervised learning, RL does not receive labeled "correct" actions for each state. Instead, the agent must balance exploration and exploitation while estimating the long-term value of its choices. Among the classical RL algorithms introduced in the course, Q-Learning is one of the most important model-free temporal-difference methods because it directly learns an action-value function and converges to the optimal policy in finite Markov decision processes under standard assumptions [1, 2].

This project follows the course implementation track and focuses on the pre-approved topic *Game Playing with Q-Learning*. We instantiate the task using FrozenLake, a compact grid-world benchmark that is widely used for teaching RL because the state space is

discrete, the policy is interpretable, and both deterministic and stochastic dynamics can be studied without requiring deep neural networks. The main objective is not only to produce a working implementation, but also to analyze how environment stochasticity and hyperparameter choice affect learning behavior.

The project has three goals. First, we build a runnable tabular Q-Learning system with training, evaluation, logging, visualization, and command-line interfaces. Second, we compare performance in deterministic and stochastic FrozenLake settings to show how transition uncertainty changes both learning speed and final policy quality. Third, we perform a small ablation study over the learning rate $\alpha$ to understand its effect on convergence stability and asymptotic performance.

# 2 Problem Definition

## 2.1 FrozenLake Environment

FrozenLake is a $4 \times 4$ grid-world with 16 discrete states. The agent starts at the top-left cell and attempts to reach the goal at the bottom-right cell while avoiding holes. Each episode terminates when the agent reaches the goal, falls into a hole, or hits the maximum step limit. The reward function is sparse: the agent receives reward 1 only when the goal is reached, and 0 otherwise. The map used in this project is

$$\begin{array}{cccc} S & F & F & F \\ F & H & F & H \\ F & F & F & H \\ H & F & F & G \end{array}$$

where $S$ denotes the start state, $F$ safe frozen cells, $H$ terminal holes, and $G$ the goal.

The action space contains four discrete actions: up, right, down, and left. We study two transition settings:

- **Deterministic setting**: the chosen action is executed exactly.
- **Stochastic setting**: the surface is slippery, so the intended direction and its left/right deviations are sampled with equal probability.

The deterministic case isolates whether the implementation can learn the shortest safe path. The stochastic case is more challenging because the agent must learn a policy that is robust to random slippage, not just a nominal shortest path.

## 2.2 RL Formulation

FrozenLake can be written as a finite Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the set of 16 states and $\mathcal{A}$ is the set of 4 actions. The transition kernel $P(s'|s, a)$ is deterministic or stochastic depending on whether the lake is slippery. The reward function $R(s, a, s')$ returns 1 only when $s'$ is the goal state and 0 otherwise. We use a discount factor $\gamma = 0.99$ to emphasize long-term return while keeping episodes finite.

The learning target is the optimal action-value function

$$Q^\star(s,a) = \max_\pi \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right].$$

Once $Q^\star$ is approximated, the greedy policy $\pi(s) = \arg\max_a Q(s,a)$ gives the best action under the learned table.

# 3 Methodology

## 3.1 Q-Learning Update

We use tabular Q-Learning with an $\epsilon$-greedy behavior policy. The Q-table has size $16 \times 4$ and is initialized to zeros. During training, with probability $\epsilon$ the agent takes a random action; otherwise it selects the greedy action with the largest Q-value. After every transition, the action value is updated by

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha \left[ r_t + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t) \right].$$

If the episode terminates at step $t+1$, the bootstrap term is omitted and the target becomes simply $r_t$.

The exploration rate starts at $\epsilon_0 = 1.0$ and decays multiplicatively after each episode until it reaches the floor $\epsilon_{\min} = 0.01$. This design encourages broad exploration early in training and more stable policy exploitation later.

## 3.2 Implementation Design

The main program `q_learning_game.py` contains the entire RL pipeline: environment construction, Q-table logic, $\epsilon$-greedy action selection, training, evaluation, plotting, and CSV export. The script supports three environment backends:

- `gymnasium`: use the standard `FrozenLake-v1` benchmark.
- `native`: use a custom FrozenLake implementation with the same map and transition semantics.
- `auto`: try Gymnasium first and fall back to the native environment if the package is unavailable.

In the actual experiments reported here, the `test` conda environment did not provide `gymnasium`, so all recorded runs used the native backend. This does not change the task definition; it only improves reproducibility by removing a dependency bottleneck while preserving the same state space, reward structure, and slippery dynamics.

### 3.3 Reproducibility Considerations

The implementation exports three kinds of artifacts:

1. **Figures**: learning curves, evaluation curves, policy visualizations, and alpha comparisons.
2. **Tables**: aggregate summaries and per-run summaries.
3. **Logs**: episode-wise training and evaluation CSV files for each seed.

The full experiment suite with 5 seeds and 10,000 episodes per run required approximately 467 seconds (7 minutes 47 seconds) on CPU in the `test` environment. This keeps the project lightweight enough for local reproduction while still being substantial enough to demonstrate stable RL trends.

## 4 Experimental Setup

### 4.1 Hyperparameters

The default hyperparameters are listed in Table 1. These values were selected to be simple, interpretable, and aligned with standard textbook Q-Learning settings for small discrete environments.

Table 1: Default hyperparameters used in the experiments.

| Symbol | Meaning | Value |
|---|---|---|
| $\alpha$ | learning rate (baseline) | 0.10 |
| $\gamma$ | discount factor | 0.99 |
| $\epsilon_0$ | initial exploration rate | 1.00 |
| $\epsilon_{\min}$ | minimum exploration rate | 0.01 |
| $\epsilon_{\text{decay}}$ | per-episode multiplicative decay | 0.995 |
| $N$ | training episodes per run | 10,000 |
| $T_{\max}$ | maximum steps per episode | 100 |
| $K$ | evaluation interval (episodes) | 500 |
| $M$ | evaluation episodes per checkpoint | 100 |
| $S$ | number of random seeds per condition | 5 |

### 4.2 Experiments

We run three core experiments:

1. **Experiment 1**: deterministic FrozenLake with `is_slippery=False`.
2. **Experiment 2**: stochastic FrozenLake with `is_slippery=True`.
3. **Experiment 3**: stochastic FrozenLake with learning-rate ablation $\alpha \in \{0.01, 0.10, 0.50\}$.

Table 2: Aggregate results over five seeds.

| Condition | $\alpha$ | Mean final success | Std. final success | Mean best success | Mean reward | Mean convergence episode |
|---|---|---|---|---|---|---|
| Exp1 deterministic baseline | 0.10 | 1.000 | 0.000 | 1.000 | 0.961 | 500 |
| Exp2 stochastic baseline | 0.10 | 0.658 | 0.084 | 0.762 | 0.461 | 6000 |
| Exp3 alpha ablation | 0.01 | 0.062 | 0.010 | 0.102 | 0.056 | — |
| Exp3 alpha ablation | 0.10 | 0.658 | 0.084 | 0.762 | 0.461 | 6000 |
| Exp3 alpha ablation | 0.50 | 0.696 | 0.012 | 0.782 | 0.583 | 1000 |

Each condition is trained independently from scratch. To reduce the risk of over-interpreting a lucky or unlucky run, every condition is repeated over five seeds. The final success rate is measured by greedy evaluation over 100 episodes. In addition to final performance, we track best evaluation success rate, mean training reward, and a practical convergence episode defined as the first evaluation checkpoint whose success rate exceeds the target threshold (0.95 for the deterministic task and 0.65 for the stochastic task).

# 5 Results and Analysis

## 5.1 Quantitative Summary

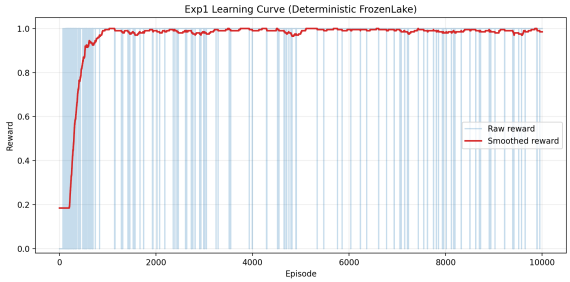
Table 2 summarizes the aggregate results across the five seeds. The deterministic baseline is solved reliably, while the stochastic environment is substantially harder. The alpha ablation shows that a higher learning rate of 0.50 works best in this project.

Three immediate conclusions follow from Table 2. First, the deterministic task is easy for tabular Q-Learning: all five seeds reach perfect final success. Second, stochastic transitions lower the attainable performance and increase variance because slippage can turn an otherwise good decision into a failure. Third, the learning-rate choice matters strongly. An overly small value ($\alpha = 0.01$) learns too slowly, while $\alpha = 0.50$ improves both convergence speed and final performance.

## 5.2 Deterministic vs. Stochastic Learning

Figure 1 compares the core baseline curves. In the deterministic environment, the reward curve rises quickly and stabilizes near one. This is consistent with the fact that the optimal path is fixed and can be discovered without transition noise. In the stochastic environment, however, reward progression is slower and the evaluation curve saturates at a lower level. Even after the policy has improved substantially, random slippage still causes some episodes to end in holes.

The five-seed comparison in Figure 2 highlights the same trend more rigorously. The deterministic curve is omitted there because it quickly becomes trivial; the stochastic baseline is the more informative case. The wide band early in training indicates that some seeds find a competent policy much earlier than others. Nonetheless, the mean performance rises steadily, confirming that the algorithm is learning useful value estimates rather than behaving randomly.




(a) Single-seed learning curve in deterministic FrozenLake.


(b) Greedy evaluation success rate in stochastic FrozenLake.

Figure 1: Baseline learning behavior under deterministic and stochastic transitions.

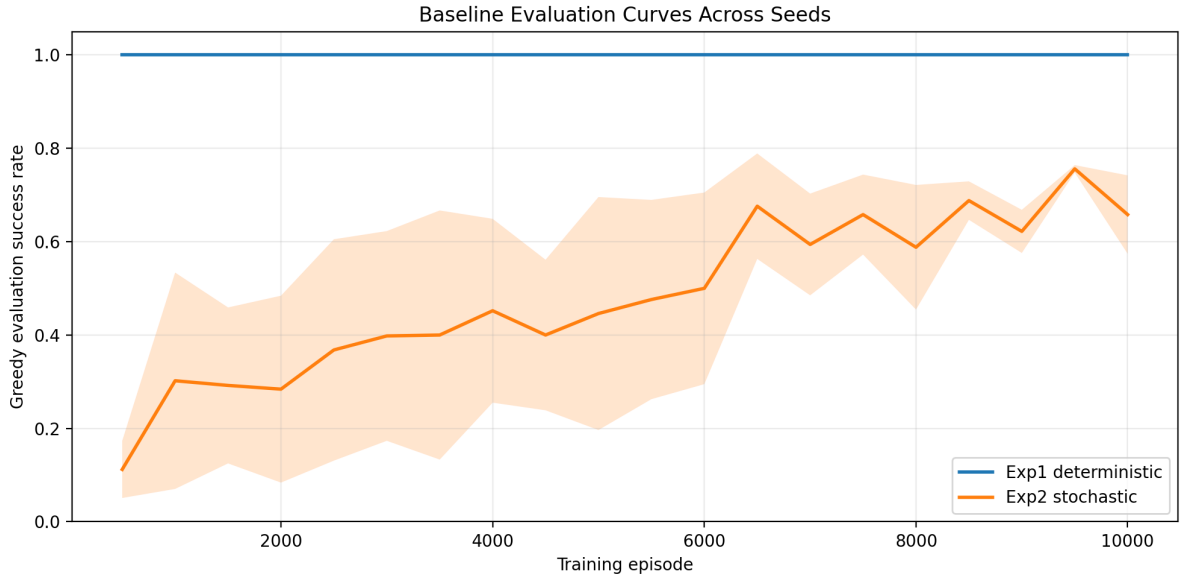
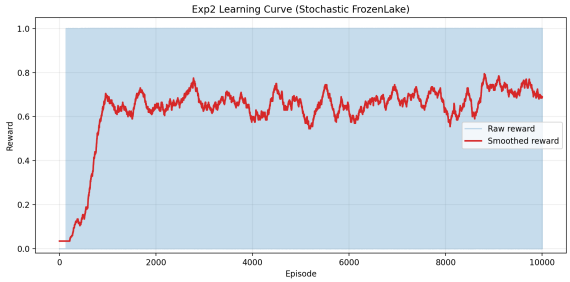



Figure 2: Mean evaluation success rate with standard-deviation band for the baseline conditions across seeds.

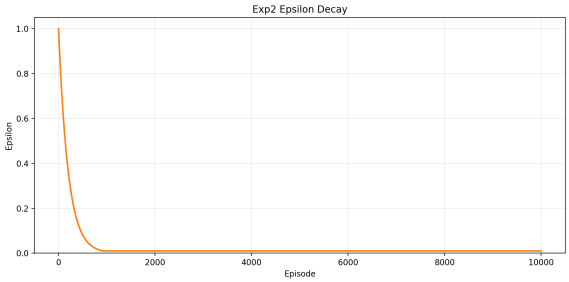
Figure 3 provides a closer look at the stochastic baseline from a single seed. The learning curve remains noisy because sparse rewards and slippery transitions occasionally produce long stretches of unsuccessful episodes even after the policy has improved. At the same time, the $\epsilon$ schedule decays smoothly from 1.0 to the minimum floor, showing that exploration is reduced in a controlled manner rather than being disabled abruptly. This combination explains why the reward trajectory is jagged while the evaluation success rate still trends upward over the long run.

## 5.3 Interpretability of the Learned Policy

A major advantage of tabular RL on FrozenLake is interpretability. Figure 4 visualizes the learned max-Q values and the greedy action in each cell for the stochastic baseline. The heatmap shows that values increase as the agent approaches the goal while avoiding high-risk hole states. The policy arrows trace a plausible route from the start toward

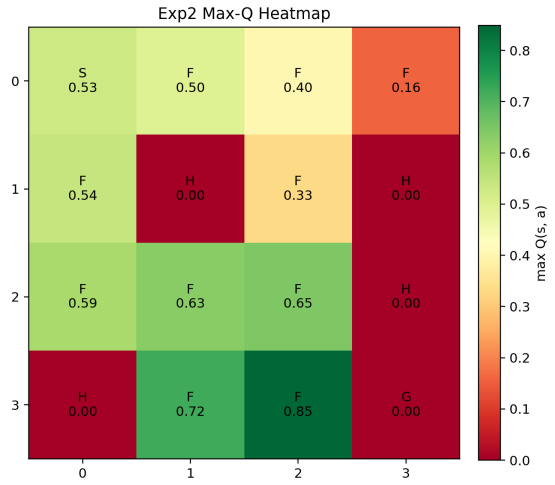



(a) Single-seed stochastic learning curve with raw and smoothed rewards.

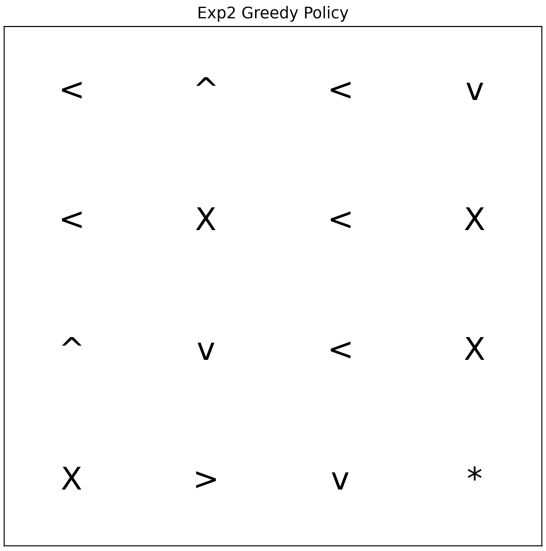



(b) Exploration schedule used during training.

Figure 3: Training dynamics for the stochastic baseline.




(a) Max-Q value heatmap for the stochastic baseline.




(b) Greedy policy derived from the learned Q-table.

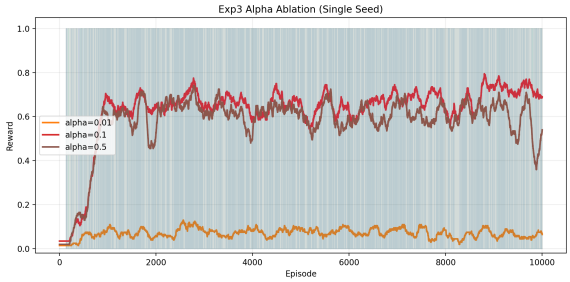
Figure 4: Interpretable value and policy visualizations for the stochastic baseline.

the goal that balances progress with safety. This provides qualitative evidence that the Q-table is not merely memorizing random rewards; it captures the structure of the task.

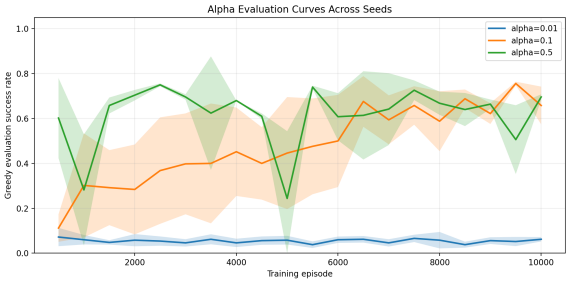
## 5.4 Learning-Rate Ablation

Figure 5 and Figure 6 summarize the alpha ablation. The single-seed curve already suggests that $\alpha = 0.01$ learns too conservatively. In contrast, both $\alpha = 0.10$ and $\alpha = 0.50$ can reach strong stochastic performance, but the larger learning rate improves more quickly.

The final-outcome plots reveal an important nuance. The bar chart in Figure 6(a) summarizes mean final success with standard-deviation error bars, while the boxplot in Figure 6(b) shows the full seed distribution. The boxplot is especially helpful because it makes the variance pattern explicit: $\alpha = 0.10$ has one noticeably weak seed, which enlarges the spread, whereas $\alpha = 0.50$ produces a tighter cluster of strong results. Therefore, the

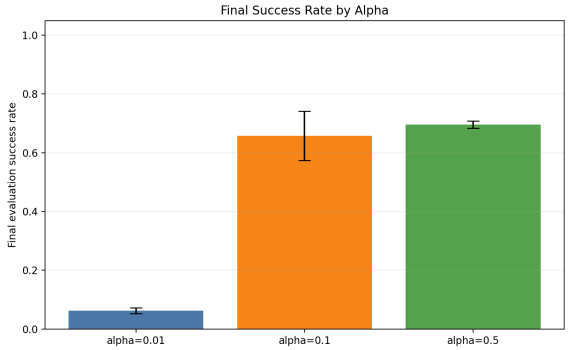



(a) Single-seed alpha comparison using smoothed reward curves.

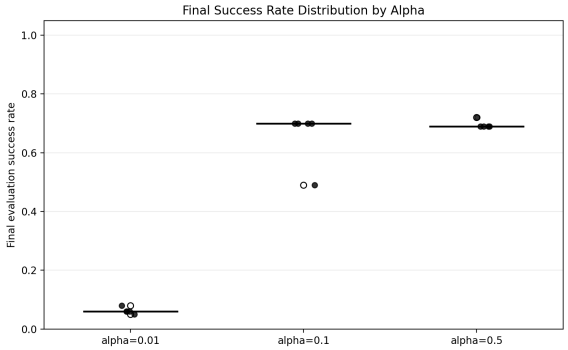



(b) Mean evaluation curves across five seeds.

Figure 5: Learning-rate ablation under stochastic dynamics.




(a) Mean final success rate with standard-deviation error bars.




(b) Seed-wise distribution shown as boxplots with scatter points.

Figure 6: Two complementary visualizations of the final alpha-ablation outcomes.

best-performing setting in this project is not only slightly better on average, but also more consistent.

## 5.5 Seed-Wise Variability and Failure Modes

Although the aggregate averages are informative, the seed-level results provide additional insight into why the stochastic setting is difficult. Under the deterministic baseline, every seed converges to perfect final success and does so very early, which means the environment structure is simple enough that exploration is sufficient to discover the optimal path quickly. Under stochastic dynamics, however, the same policy can produce different returns depending on accidental slips into holes or detours away from the shortest route. This causes the training curves to remain noisy even after the Q-values have become qualitatively reasonable.

The seed-level records also show that the weakest stochastic baseline run finished at 0.49 final evaluation success, while the other four baseline runs ended around 0.70. This gap does not mean that Q-Learning fundamentally failed on one seed; rather, it suggests that the interaction between sparse rewards, decaying exploration, and stochastic transitions can trap the learner in a suboptimal value estimate for part of training. This observation is precisely why multi-seed reporting is preferable to single-run reporting in RL experiments.

If only the weak seed had been shown, the baseline would have looked substantially worse than its typical behavior.

For the alpha ablation, the contrast is even clearer. The smallest learning rate, $\alpha = 0.01$, updates the table too cautiously and therefore underfits the task within 10,000 episodes. Its final success rates remain clustered near zero, indicating that the learner often fails to propagate useful goal information backward through the state space. In contrast, $\alpha = 0.50$ produces a compact cluster of strong results around 0.69–0.72. This suggests that faster value propagation is advantageous in FrozenLake because the state-action space is tiny, the rewards are sparse, and the bootstrap target is still stable enough to avoid divergence.

# 6 Discussion

The experimental results align with standard RL intuition. Sparse-reward deterministic tasks are often enough for vanilla Q-Learning when the state space is small and fully observable. Once stochasticity is introduced, learning becomes slower and more sensitive to random trajectories because failures can happen even after a good action is chosen. This is exactly what we observe in the stochastic baseline.

The alpha ablation also highlights a practical lesson for RL implementation. A lower learning rate is not automatically safer or better. In this task, $\alpha = 0.01$ updates the value table so slowly that exploration noise dominates the learning process for most of training. Meanwhile, $\alpha = 0.50$ is large enough to adapt quickly without causing instability in the tiny tabular setting. For larger or function-approximation-based problems, such a high learning rate might be risky, but here it is beneficial.

One limitation of the study is that it uses a small benchmark with only 16 states, so conclusions should not be over-generalized to complex environments. Another limitation is that the project focuses on Q-Learning only; a stronger comparison could include SARSA or expected SARSA under the same experimental protocol. Nevertheless, the current scope is appropriate for a course project because it clearly demonstrates core RL concepts, interpretable policies, and hyperparameter sensitivity in a reproducible setting.

From a software-engineering perspective, the project also shows why reproducibility matters in RL coursework. The implementation writes all figures and CSV artifacts automatically, supports command-line selection of experiments and seeds, and keeps the environment layer interchangeable through the `auto`, `gymnasium`, and `native` backends. This design reduced the risk of tool-chain failure affecting the scientific conclusions. In practice, the native fallback was especially useful because the benchmark package was not available in the active execution environment, yet the experiments could still be reproduced faithfully. The final pipeline therefore satisfies the course expectation of runnable code together with clear reproduction instructions.

Finally, the experiments illustrate a broader methodological lesson: single-seed RL results can be misleading. In the stochastic baseline, one seed under $\alpha = 0.10$ ended substantially below the others, which would have led to a pessimistic conclusion if only one run had been reported. Repeating each condition five times provided a better estimate of both average performance and stability. The added boxplot for the alpha-ablation experiment makes this point visually clear and strengthens the reliability of the final recommendation.

### 6.1 Threats to Validity and Future Extensions

Several threats to validity should be acknowledged explicitly. FrozenLake is a small tabular benchmark, so its conclusions do not automatically transfer to large-scale or continuous-control problems. In addition, only one exploration schedule was tested, and the custom native backend—although designed to match the benchmark definition—may still differ slightly from library implementations in minor low-level behavior. For these reasons, the present report emphasizes robust qualitative trends and aggregate multi-seed outcomes rather than exact benchmark matching at the last decimal place.

If the project were extended, the most natural next steps would be algorithmic comparison and broader hyperparameter analysis. SARSA or expected SARSA could be tested under the same protocol, while larger maps such as $8 \times 8$ FrozenLake would make exploration substantially harder. A wider study over $\gamma$, the minimum exploration floor, and the number of seeds would also strengthen the statistical interpretation of the results.

## 7 Conclusion

This project successfully implements a complete Q-Learning pipeline for FrozenLake and evaluates it under both deterministic and stochastic settings. The deterministic baseline is solved perfectly across all seeds, confirming the correctness of the implementation. The stochastic baseline remains challenging but reaches a mean final success rate of 0.658, showing that tabular Q-Learning can still discover a competent policy under transition noise. The learning-rate ablation further shows that $\alpha = 0.50$ is the strongest setting in this project, outperforming the default baseline and converging much faster.

Beyond the raw scores, the project also emphasizes reproducibility and interpretability. The implementation exports logs, tables, and visualization assets; it supports a native environment fallback; and it generates interpretable heatmaps and policy diagrams. These characteristics make the project suitable not only for submission, but also for later presentation and discussion. Future work could extend the study to larger maps, additional RL algorithms, or a more systematic hyperparameter search, but the present implementation already satisfies the course requirement of a runnable RL system with clear results and analysis.

The most important practical takeaway is that even a classical tabular RL algorithm can exhibit meaningful stability differences across hyperparameter settings once stochasticity is introduced. In this project, the best recommendation is to prefer $\alpha = 0.50$ over the default baseline for the slippery environment, because it converges earlier and produces the most stable final performance across seeds. More broadly, the project demonstrates that careful experiment design, repeated trials, and interpretable visual analysis are just as important as obtaining a single headline score. Those aspects are central to doing reinforcement learning well in both coursework and research settings.

# References

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[3] Farama Foundation, "Gymnasium documentation," 2023. [Online]. Available: https://gymnasium.farama.org/

# A Reproduction Commands

The full project can be reproduced with the following command in the project root:

```
conda activate test
python q_learning_game.py --exp all --multi-seed 5 --output-dir results
```

The report was compiled from the self-contained LaTeX source located in the `report/` directory. An Overleaf-ready archive is also provided so that the PDF can be rebuilt without relying on the local Python project structure.

# B Selected Per-Seed Results

| Condition | Seed | Final success | Best success | Convergence episode |
|---|---|---|---|---|
| Exp1 baseline | 0 | 1.00 | 1.00 | 500 |
| Exp1 baseline | 1 | 1.00 | 1.00 | 500 |
| Exp1 baseline | 2 | 1.00 | 1.00 | 500 |
| Exp1 baseline | 3 | 1.00 | 1.00 | 500 |
| Exp1 baseline | 4 | 1.00 | 1.00 | 500 |
| Exp2 baseline | 0 | 0.70 | 0.77 | 1000 |
| Exp2 baseline | 1 | 0.49 | 0.75 | 8500 |
| Exp2 baseline | 2 | 0.70 | 0.75 | 9000 |
| Exp2 baseline | 3 | 0.70 | 0.78 | 5000 |
| Exp2 baseline | 4 | 0.70 | 0.76 | 6500 |
| Exp3 alpha 0.01 | 0 | 0.08 | 0.13 | — |
| Exp3 alpha 0.01 | 1 | 0.05 | 0.08 | — |
| Exp3 alpha 0.01 | 2 | 0.06 | 0.12 | — |
| Exp3 alpha 0.01 | 3 | 0.06 | 0.10 | — |
| Exp3 alpha 0.01 | 4 | 0.06 | 0.08 | — |
| Exp3 alpha 0.50 | 0 | 0.69 | 0.78 | 1500 |
| Exp3 alpha 0.50 | 1 | 0.69 | 0.77 | 500 |
| Exp3 alpha 0.50 | 2 | 0.69 | 0.78 | 500 |
| Exp3 alpha 0.50 | 3 | 0.69 | 0.80 | 500 |
| Exp3 alpha 0.50 | 4 | 0.72 | 0.78 | 2000 |